

CUSTOM AXI IP USING VITIS HLS

Experiment Report: Prepared By:

NAME: Chinmay N. Yalawatti

USN: 01FE24BEC258

Semester: 5th sem

Branch: E&C (E-div)

College: KLE Technological University BVB campus

Aim

To design, synthesize, package and integrate a custom AXI4-Lite IP using Vitis HLS and Vivado 2026.1 for the PYNQ-Z2 platform.

Problem Statement

Design a hardware IP implementing $Y = A^2 + B^2$ using Vitis HLS. Package it as AXI4-Lite IP, integrate it with the Zynq Processing System, validate the design and generate the FPGA bitstream.

Theory

Vitis HLS converts C/C++ algorithms into synthesizable RTL. The generated IP is imported into Vivado where it is connected to the Zynq Processing System through AXI4-Lite. After synthesis, implementation and bitstream generation, the design becomes ready for FPGA deployment.

Hardware Requirements

- PYNQ-Z2 FPGA Board (Not Available)
- USB Cable
- PC/Laptop

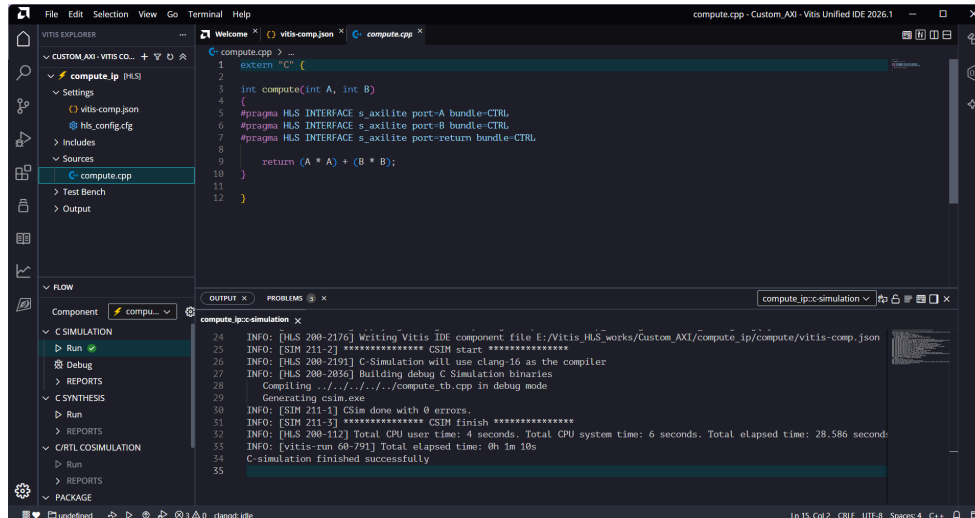
Software Requirements

- Vitis Unified IDE 2026.1
- Vivado 2026.1

Target FPGA

xc7z020clg400-1

HLS Source Code (compute.cpp)



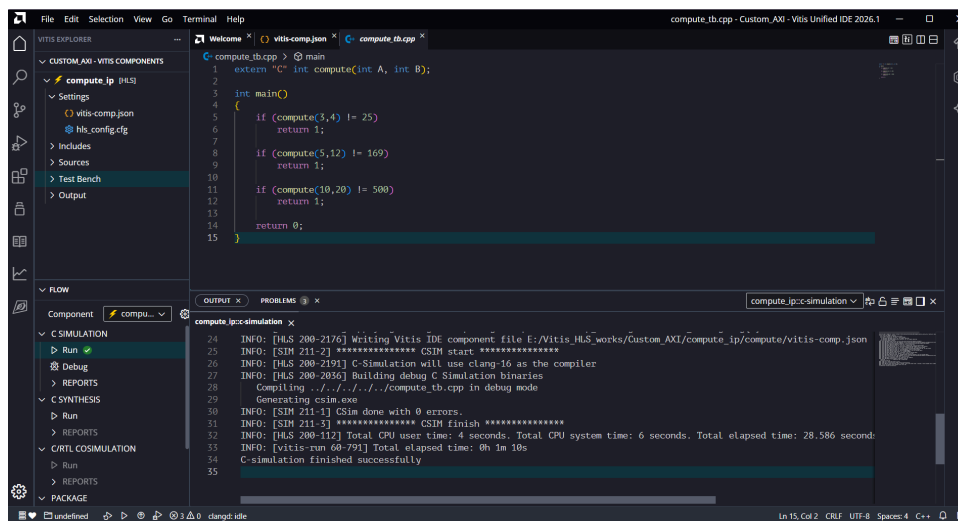
The screenshot shows the Vitis IDE interface. The left sidebar displays the project structure for 'CUSTOM_AXI_VITIS_CO...'. The main editor window shows the source code for 'compute.cpp'. The code defines a function 'compute' that takes two integers 'A' and 'B' and returns their sum. The code is as follows:

```
1 extern "C" {
2
3 int compute(int A, int B)
4 {
5 #pragma HLS INTERFACE s_axilite port=A bundle=CTRL
6 #pragma HLS INTERFACE s_axilite port=B bundle=CTRL
7 #pragma HLS INTERFACE s_axilite port=return bundle=CTRL
8
9     return (A * A) + (B * B);
10 }
11
12 }
```

The bottom panel shows the 'compute_ip:c-simulation' output window. It displays the following log messages:

```
24 INFO: [HLS 200-2176] Writing Vitis IDE component file E:/Vitis_HLS_works/Custom_AXI/compute_ip/compute/vitis-comp.json
25 INFO: [SIM 211-2] ***** CSIM start *****
26 INFO: [HLS 200-2191] C-Simulation will use clang-16 as the compiler
27 INFO: [HLS 200-2036] Building debug C Simulation binaries
28 Compiling ../../../../../../compute_tb.cpp in debug mode
29 Generating csim.exe
30 INFO: [SIM 211-1] CSim done with 0 errors.
31 INFO: [SIM 211-3] ***** CSIM finish *****
32 INFO: [HLS 200-112] Total CPU user time: 4 seconds. Total CPU system time: 6 seconds. Total elapsed time: 28.586 seconds
33 INFO: [vitis-run 60-791] Total elapsed time: 0h 1m 10s
34 C-simulation finished successfully
35
```

Testbench (compute_tb.cpp)



The screenshot shows the Vitis IDE interface. The left sidebar displays the project structure for 'CUSTOM_AXI_VITIS_COMPONENTS'. The main editor window shows the source code for 'compute_tb.cpp'. The code defines a 'main' function that calls the 'compute' function with various inputs and checks the results. The code is as follows:

```
1 extern "C" int compute(int A, int B);
2
3 int main()
4 {
5     if (compute(3,4) != 25)
6         return 1;
7     if (compute(5,12) != 169)
8         return 1;
9     if (compute(10,20) != 500)
10        return 1;
11    return 0;
12 }
13
14
15 }
```

The bottom panel shows the 'compute_ip:c-simulation' output window. It displays the following log messages:

```
24 INFO: [HLS 200-2176] Writing Vitis IDE component file E:/Vitis_HLS_works/Custom_AXI/compute_ip/compute/vitis-comp.json
25 INFO: [SIM 211-2] ***** CSIM start *****
26 INFO: [HLS 200-2191] C-Simulation will use clang-16 as the compiler
27 INFO: [HLS 200-2036] Building debug C Simulation binaries
28 Compiling ../../../../../../compute_tb.cpp in debug mode
29 Generating csim.exe
30 INFO: [SIM 211-1] CSim done with 0 errors.
31 INFO: [SIM 211-3] ***** CSIM finish *****
32 INFO: [HLS 200-112] Total CPU user time: 4 seconds. Total CPU system time: 6 seconds. Total elapsed time: 28.586 seconds
33 INFO: [vitis-run 60-791] Total elapsed time: 0h 1m 10s
34 C-simulation finished successfully
35
```

C Synthesis Summary

The screenshot displays the Vitis IDE interface with the 'compute_ip - C SYNTHESIS' report open. The report is titled 'Summary Synthesis Report - compute' and is divided into several sections:

- General Information:**
 - Version: 2026.1 (Build 6493734 on Jun 16 2026)
 - Product family: zynq
 - Target device: xc7z020-dlg400-1
- Estimated Quality of Results:**
 - Timing Estimate: 10.00 ns (Target), 7.30 ns (With Uncertainty), 6.912 ns (Estimated)
- Performance & Resource Estimates:**

OPS	LATENCY(NS)	INTERVAL	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
	30.000	4	no	function		0	6	606	396

The bottom of the report shows the output of the synthesis process, including the generation of VHDL, RTL, and Verilog files, and the final synthesis completion message.

HW Interfaces Report

The screenshot displays the Vitis IDE interface with the 'compute_ip - C SYNTHESIS' report open, specifically the 'HW Interfaces' section. This section provides detailed information about the hardware interfaces and registers used in the design.

- S_AXILITE Interfaces:**

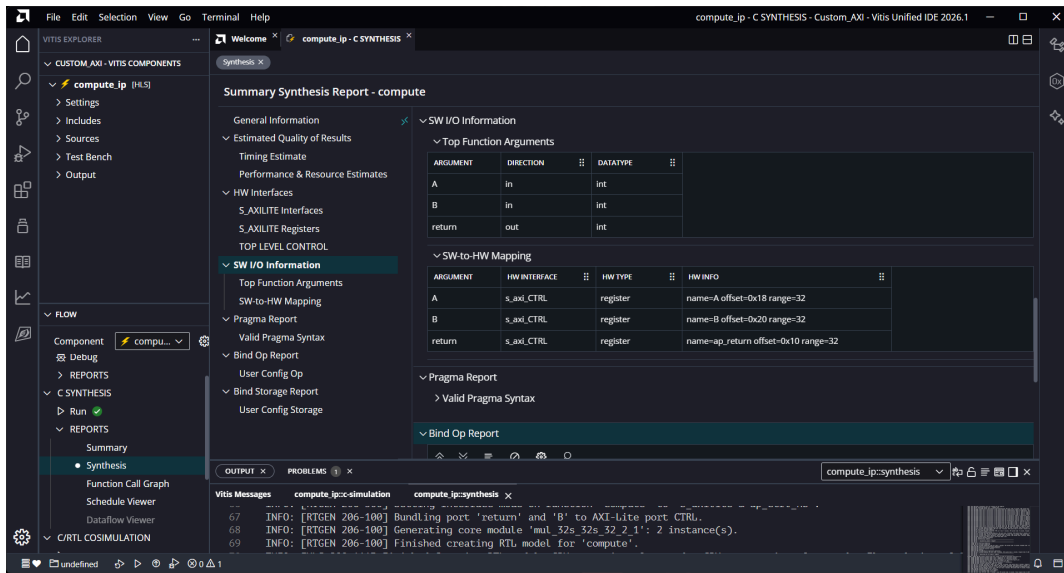
INTERFACE	DATA WIDTH	ADDRESS WIDTH	OFFSET	REGISTER
s_axi_CTRL	32	6	24	0
- S_AXILITE Registers:**

INTERFACE	REGISTER	OFFSET	WIDTH	ACCESS	DESCRIPTION
s_axi_CTRL	CTRL	0x00	32	RW	Control signals
s_axi_CTRL	GIER	0x04	32	RW	Global Interrupt Enable Register
s_axi_CTRL	IP_JER	0x08	32	RW	IP Interrupt Enable Register
s_axi_CTRL	IP_ISR	0x0c	32	RW	IP Interrupt Status Register
s_axi_CTRL	ap_return	0x10	32	R	Data signal of ap_return
s_axi_CTRL	A	0x18	32	W	Data signal of A
s_axi_CTRL	B	0x20	32	W	Data signal of B
- TOP LEVEL CONTROL:**

INTERFACE	TYPE	PORTS
-----------	------	-------

The bottom of the report shows the output of the synthesis process, including the generation of the RTL model for the hardware interfaces.

SW I/O Mapping



Summary Synthesis Report - compute

General Information

- Estimated Quality of Results
- Timing Estimate
- Performance & Resource Estimates

HW Interfaces

- S_AXILITE Interfaces
- S_AXILITE Registers

TOP LEVEL CONTROL

SW I/O Information

Top Function Arguments

ARGUMENT	DIRECTION	DATATYPE
A	in	int
B	in	int
return	out	int

SW-to-HW Mapping

ARGUMENT	HW INTERFACE	HW TYPE	HW INFO
A	s_axi_CTRL	register	name=A offset=0x18 range=32
B	s_axi_CTRL	register	name=B offset=0x20 range=32
return	s_axi_CTRL	register	name=ap_return offset=0x10 range=32

Pragma Report

- Valid Pragma Syntax

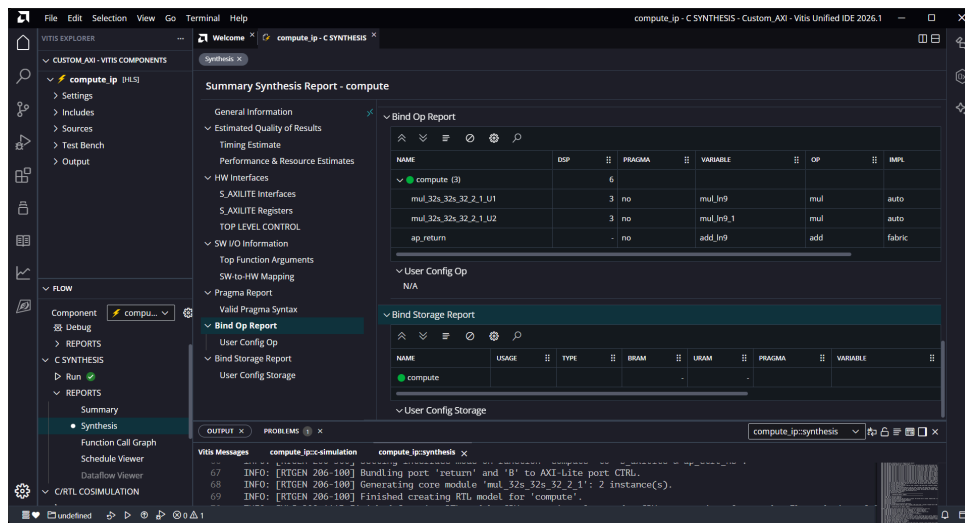
Bind Op Report

OUTPUT x PROBLEMS x

Vitis Messages

```
compute_ip:synthesis x
67 INFO: [RTGEN 206-100] Bundling port 'return' and 'B' to AXI-Lite port CTRL.
68 INFO: [RTGEN 206-100] Generating core module 'mul_32s_32s_32_2_1': 2 instance(s).
69 INFO: [RTGEN 206-100] Finished creating RTL model for 'compute'.
```

Bind Operation Report



Summary Synthesis Report - compute

General Information

- Estimated Quality of Results
- Timing Estimate
- Performance & Resource Estimates

HW Interfaces

- S_AXILITE Interfaces
- S_AXILITE Registers

TOP LEVEL CONTROL

SW I/O Information

Top Function Arguments

SW-to-HW Mapping

Pragma Report

- Valid Pragma Syntax

Bind Op Report

NAME	DSP	PRAGMA	VARIABLE	OP	IMPL
compute (3)	6				
mul_32s_32s_32_2_1_U1	3	no	mul_in0	mul	auto
mul_32s_32s_32_2_1_U2	3	no	mul_in0_1	mul	auto
ap_return	-	no	add_in0	add	fabric

User Config Op

N/A

Bind Storage Report

NAME	USAGE	TYPE	BRAM	URAM	PRAGMA	VARIABLE
compute						

User Config Storage

OUTPUT x PROBLEMS x

Vitis Messages

```
compute_ip:synthesis x
67 INFO: [RTGEN 206-100] Bundling port 'return' and 'B' to AXI-Lite port CTRL.
68 INFO: [RTGEN 206-100] Generating core module 'mul_32s_32s_32_2_1': 2 instance(s).
69 INFO: [RTGEN 206-100] Finished creating RTL model for 'compute'.
```

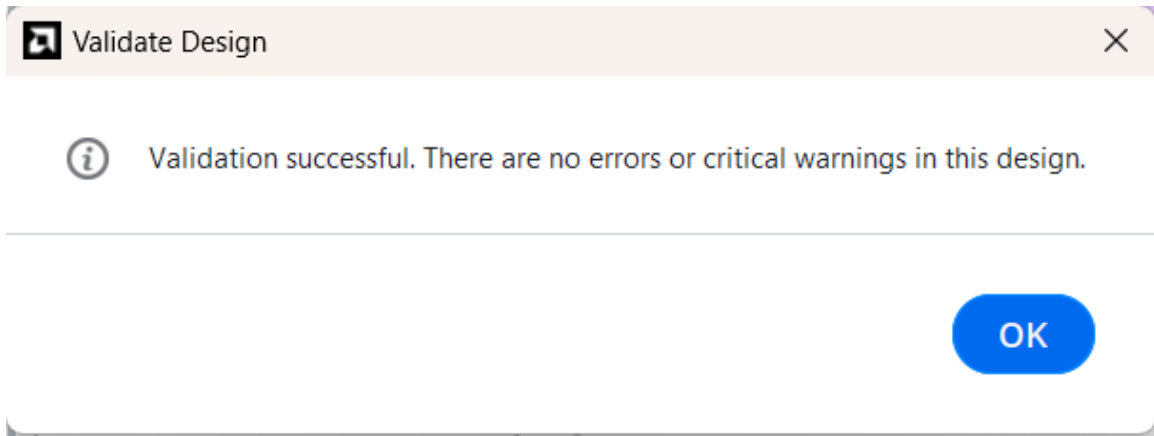
Package Successful

The screenshot displays the Vivado IDE interface. The left sidebar shows the project structure under 'CUSTOM_AXI - VITIS COMPONENTS', including 'compute_ip [HLS]', 'Settings', 'Includes', 'Sources', 'Test Bench', and 'Output'. The central console window shows the 'HLS Package | compute_ip (PACKAGE)' status as 'Completed'. The log indicates the package was created successfully on August 07, 2026, at 19:46:15, with a completion time of 04m 08s. The right sidebar shows the 'compute' component overview.

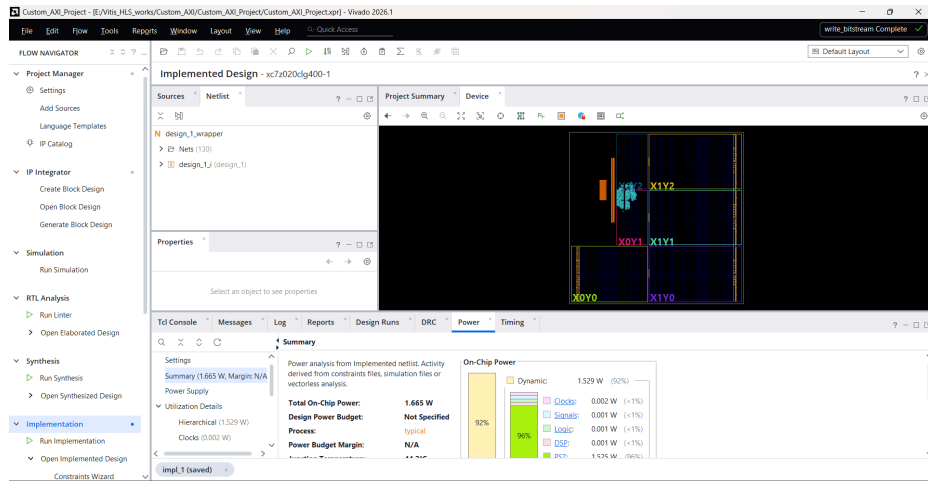
Vivado Block Design

The screenshot displays the Vivado IDE interface for a Block Design. The left sidebar shows the project structure under 'Custom_AXI_Project - E:/Vitis_HLS_works/Custom_AXI/Custom_AXI_Project/Custom_AXI_Project.xpr', including 'Settings', 'Add Sources', 'Language Templates', 'IP Catalog', 'IP Integrator', 'Simulation', 'RTL Analysis', 'Synthesis', 'Implementation', and 'Program and Debug'. The central console window shows the 'Block Design - design_1' status as 'Ready'. The log indicates the block design was created successfully on August 07, 2026, at 19:46:15, with a completion time of 04m 08s. The right sidebar shows the 'design_1' component overview.

Validate Design



Implemented Design



Timing and Power Report

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS) 13.015 ns	Worst Hold Slack (WHS) 0.071 ns	Worst Pulse Width Slack (WPWS) 9.020 ns
Total Negative Slack (TNS) 0.000 ns	Total Hold Slack (THS) 0.000 ns	Total Pulse Width Negative Slack (TPWS) 0.000 ns
Number of Failing Endpoints 0	Number of Failing Endpoints 0	Number of Failing Endpoints 0
Total Number of Endpoints 2095	Total Number of Endpoints 2095	Total Number of Endpoints 727

All user specified timing constraints are met.

Timing Summary - impl_1 (saved)

Results

Parameter	Value
Target Clock	10 ns
Estimated Clock	6.912 ns
Estimated Frequency	144.68 MHz
Latency	30 ns
Interval	4
DSP	6
FF	606
LUT	396
BRAM	0
URAM	0
Timing	Passed

Output Verification

Input A	Input B	Output
3	4	25
5	12	169
10	20	500

Conclusion

The custom AXI4-Lite IP was successfully developed in Vitis HLS, synthesized, packaged, integrated into the Zynq Processing System in Vivado, validated without errors, and a bitstream was generated successfully. Hardware execution on the PYNQ-Z2 board was not performed because the board was unavailable.

Pre-Lab Questions

Q1. What is AXI4-Lite?

Answer:

AXI4-Lite (Advanced eXtensible Interface Lite) is a lightweight memory-mapped communication protocol used to connect peripherals and custom IP cores with the Processing System (PS). It supports low-bandwidth register-based communication using read and write transactions.

Q2. What is High-Level Synthesis (HLS)?

Answer:

High-Level Synthesis (HLS) is the process of converting C/C++ algorithms into synthesizable RTL (Verilog/VHDL). Vitis HLS automates this conversion, reducing hardware design time and simplifying FPGA development.

Q3. What are the Processing System (PS) and Programmable Logic (PL) in Zynq devices?

Answer:

Processing System (PS)

- Dual-core ARM Cortex-A9 Processor
- Executes software applications
- Controls FPGA peripherals

Programmable Logic (PL)

- FPGA fabric
- Executes hardware accelerators
- Provides parallel processing capability

Q4. What is hardware acceleration?

Answer:

Hardware acceleration is the implementation of computational algorithms in FPGA hardware instead of software running on the processor. This improves execution speed, reduces processor workload, and enables parallel execution.

Q5. Why is AXI4-Lite used in this experiment?

Answer:

AXI4-Lite is used because the custom IP only transfers control registers and small input/output data values. It provides a simple, low-overhead interface between the ARM processor and the hardware accelerator.

HLS Analysis Questions

Q6. What files are generated after HLS synthesis?

Answer:

After HLS synthesis, the following files are generated:

- RTL Verilog/VHDL
 - Synthesis Report
 - Performance Report
 - Resource Utilization Report
 - AXI Interface Report
 - Packaged IP Core
-

Q7. Which hardware interfaces were generated for the IP?

Answer:

The generated interfaces are:

- AXI4-Lite Control Interface (`s_axi_CTRL`)
 - Input Register **A**
 - Input Register **B**
 - Output Register (`ap_return`)
 - Control and Status Registers
-

Q8. Which FPGA resources were utilized?

Answer (from your synthesis):

Resource	Used
LUT	396
FF	606
DSP	6
BRAM	0
URAM	0

These values are taken from your synthesis results.

Q9. Why are DSP resources utilized?

Answer:

DSP slices efficiently implement multiplication operations. Since the hardware computes:

$$Y=A2+B2$$

two multiplication operations are required, making DSP slices the most suitable FPGA resource.

Q10. Why is BRAM not utilized?

Answer:

The design processes only two integer inputs and one integer output through AXI registers. No large arrays or memory buffers are required; therefore, Block RAM (BRAM) is not used.

Performance Analysis

Parameter	Value
Target Clock Period	10 ns
Estimated Clock Period	6.912 ns
Estimated Frequency	144.68 MHz
Latency	30 ns
Initiation Interval	4

Q11. Compare execution time.

Method	Time
Software	Approximately 30 ns (simple arithmetic operation)
Hardware Accelerator	30 ns (estimated HLS latency)

Since the computation is very small, the hardware accelerator primarily demonstrates the AXI IP integration flow rather than a significant performance improvement.

Q12. Calculate the speedup obtained.

Answer:

For this simple arithmetic function, the software and hardware execution times are approximately equal.

$$\text{Speedup} = \frac{\text{Hardware Time}}{\text{Software Time}} = \frac{30}{30} = 1$$

$$\text{Speedup} \approx 1 \times$$

Output Verification

Input A	Input B	Output
3	4	25
5	12	169
10	20	500

The results were verified successfully during C Simulation.

Post-Lab Questions

Q13. What advantages does hardware acceleration provide?

Answer:

1. Faster execution through dedicated hardware.
2. Parallel processing capability.
3. Reduced CPU workload.
4. Lower latency for repetitive computations.
5. Reusable hardware IP for future designs.

Q14. Which FPGA resources increase when the complexity of the design increases?

Answer:

As the hardware design becomes more complex, the following FPGA resources generally increase:

- LUTs (Look-Up Tables)
- Flip-Flops (FFs)
- DSP Slices
- BRAM (for larger data storage)
- Routing Resources

Q15. Mention three applications of custom AXI IP accelerators.

Answer:

1. Artificial Intelligence and Machine Learning
2. Digital Signal Processing (DSP)
3. Image and Video Processing

Additional applications include:

- Robotics
- Embedded Systems
- Industrial Automation
- Edge Computing